

Secure Software Engineering - Session 1

Security Problem

- Organizations store, process, and transmit sensitive information, making them **vulnerable**.
- Citizens depend on software for banking, shopping, etc., exposing them to **cyber risks**.
- We live in the era of:
 - **Information warfare**
 - **Cyber terrorism**
 - **Computer crime**
- **Example:** Hacker groups targeting election systems, banking apps, healthcare data.

Security Principles (Saltzer & Schroeder, 1974)

- **Confidentiality:** Only authorized access is permitted.
- **Integrity:** Data is accurate and has not been modified.
- **Availability:** Reliable and timely access to the system.
- **Additional Properties:**
 - **Accountability:** Actions must be logged and traceable.
 - **Non-repudiation:** The ability to prove a party performed an action, so they cannot deny it.
- **Example:** Online banking requires confidentiality (encryption), integrity (no tampered transactions), and availability (system up 24/7).

Software Assurance

- **Definition:** The confidence that a system functions as intended and is free of exploitable vulnerabilities.
- **Includes:**
 - Software reliability
 - Software safety
 - Software security
- **Example:** Defense systems must manage risks of attacks throughout the whole lifecycle.

Software Resilience

- **Definition:** The ability to absorb, adapt, and recover quickly from disruptive events.
- A resilient system continues its mission even under attack.
- **Example:** Payment gateways must still process transactions even if part of the system is down.

Processes for Secure Software

- Secure software depends on processes including:
 - Specification & design
 - Development choices (language, tools)
 - Testing practices
- **Example:** SQL injection happens due to poor input validation in development.

Security Concepts & Relationships

- **Threat:** A potential harmful event.
- **Vulnerability:** A weakness that can be exploited by a threat.
- **Risk:** The product of likelihood and impact ($\text{Risk} = \text{Likelihood} \times \text{Impact}$).
- **Countermeasure:** Controls to prevent or mitigate attacks.
- **Defense-in-Depth:** The use of multiple security layers to protect a system.
 - **Example:** Firewall + Intrusion Detection System (IDS) + Antivirus.
- **Sandboxing:** An isolated environment to run untrusted programs safely.
 - **Example:** Running downloaded apps in a virtual sandbox.

Privacy & GDPR

- **GDPR** (General Data Protection Regulation) enforces strong data protection laws in the EU.
- Companies must support:
 - Data portability
 - User consent
 - Data impact assessments
 - Right to be forgotten
- **Example:** Facebook must allow EU users to download and delete personal data.

Threats & Attacks

- **Social engineering:** Tricking users into giving access.
- **Cloud risks:** Weak control panel access can destroy infrastructure.
 - **Example:** Code Spaces AWS attack.
- **Account chaining attacks:** One breached account leads to others.
 - **Example:** Mat Honan hack (2012) – Gmail → Twitter → AppleID.
- **Attack Surfaces:** All possible entry points for an attacker.
 - **Network:** Open ports.
 - **Software:** Bugs, insecure APIs.
 - **Human:** Insider misuse, phishing.
- **Attack Vectors:** The path used to exploit a vulnerability.
 - **Example:** An email with a malicious attachment.

Attack Trees

- A hierarchical model showing attack paths.
- Helps identify weak points and countermeasures.
- **Example:** Defacing a website could be achieved via:
 - DNS hijack
 - CMS exploit
 - CDN misconfiguration

Malware

- **Malware Definition:** Malicious software designed to harm or steal.
- **Types:**
 - **Virus:** Needs a host program to spread.
 - **Worm:** A self-replicating program.
 - **Trojan horse:** Appears legitimate but has hidden functionality.
 - **Logic bomb:** Triggers when a specific condition is met.
 - **Rootkit:** Provides a stealthy way to take over a system.
 - **Keylogger:** Steals keystrokes.
 - **Bot/Zombie:** A compromised machine used in botnets.
- **Example:** WannaCry ransomware (2017) spread like a worm.

Viruses

- **Phases:** Dormant → Propagation → Trigger → Execution.
- **Classification:**
 - **By target:** Boot sector, File infector, Macro virus.
 - **By concealment:** Encrypted, Stealth, Polymorphic, Metamorphic.
- **Example:** Melissa Virus (1999) spread via MS Word macros.

Worms

- Self-propagating programs.
- **Example:** Morris Worm (1988) exploited UNIX bugs.
- **Modern worms:** Multiplatform, zero-day exploits, polymorphic.
- **Countermeasures:**
 - Signature-based detection
 - Rate limiting abnormal traffic
 - Payload anomaly detection

Security Trends

- Vulnerabilities reported in the **NVD** (National Vulnerability Database) continue to rise.
- **India:** A surge in data leaks, ransomware, and DDoS attacks (2024).
- **Gartner:** The role of the cybersecurity leader is shifting to that of a business enabler.
- **Example:** The Colonial Pipeline ransomware attack (2021) disrupted the US fuel supply.

Supply Chain Attacks

- Occur when a software vendor is compromised before delivering to customers.
- **Examples by Lifecycle Phase:**
 - **Design:** Hijacked devices send data abroad.
 - **Development:** SolarWinds attack (2020).
 - **Distribution:** Malware preinstalled on PCs.
 - **Deployment:** Kaspersky AV ban in the US (2017).
 - **Maintenance:** Backdoors in software updates.

Summary

- The **CIA Triad** (Confidentiality, Integrity, Availability) plus Accountability & Non-repudiation are the core of security.
- **Risks** arise from **threats** exploiting **vulnerabilities**.
- **Attack surfaces** are all possible entry points.
- **Malware** and **worms** remain evolving threats.
- Secure processes, resilience, assurance, and GDPR compliance are key to secure software engineering.
- **Supply chain attacks** highlight the importance of security throughout the entire software lifecycle.

Software Engineering - Session 2

1. Software Engineering – Definitions

- **Fritz Bauer ():** Software engineering is about applying engineering principles to build reliable and efficient software on real machines.
- **IEEE Definition:** The application of a systematic, disciplined, and measurable approach to software development, operation, and maintenance.
- **Key Idea:** Treat software development like engineering — structured, planned, and measurable.
- **Example:** Just as civil engineering uses tested designs for bridges, software engineering uses processes and standards for building banking apps or healthcare systems.

2. Phases in Software Development

- Software evolves in phases: requirements, design, construction, testing, deployment, and maintenance.
- Each phase must ensure both quality and security.

3. Knowledge Areas (SWEBOK v4 – 2024)

- **Primary areas:**
 - Requirements
 - Architecture
 - Design
 - Construction
 - Testing
 - Maintenance
- **Supporting areas:**
 - Configuration Management (CM)
 - Quality
 - Security
 - Processes
 - Models & Methods
 - Engineering & Project Management

- Economics
- Professional Practice
- **New in v4:** Architecture and Security have been added as new areas.
- **Example:**
 - **Primary (core):** Building a login module involves requirements (what it should do), design (how it looks/works), construction (coding), and testing.
 - **Supporting:** This also involves ensuring version control (CM), meeting deadlines (Project Management), and checking for performance and security.

4. Software Engineering Process

- **Definition (SWEBOK):** A set of interrelated activities and tasks that transform inputs into outputs (software products).
- **Includes:** Inputs → Work Activities → Outputs
- **Lifecycle (SDLC):** The process used to move from requirements to deliverable software.
- **Example:** Agile Scrum defines roles (who), ceremonies (when), and tasks (how).

5. Process Models – Differences

- Process models differ in:
 - Flow of activities (linear vs. iterative)
 - Level of detail and rigor
 - Stakeholder involvement
 - Team autonomy and roles
 - Project tracking and control methods
- **Example:**
 - **Waterfall:** Sequential, rigid.
 - **Agile:** Iterative, flexible, with continuous feedback.

6. Prescriptive vs. Agile Processes

- **Prescriptive:** Planned fully in advance. Progress is measured against the plan.
 - **Example:** A defense project using the Waterfall model.
- **Agile:** Uses incremental planning and is adaptable to customer changes.
 - **Example:** An e-commerce startup where features evolve with user feedback.
- **Key Insight:** Most practical projects mix both approaches.

7. Software Requirements

- Concerned with the elicitation, analysis, specification, validation, and management of requirements.
- Poorly defined requirements lead to a high risk of failure.
- **Types:**
 - **Functional:** Define what the system does.
 - **Example:** “The system shall allow users to reset their password.”
 - **Non-Functional:** Define how well the system performs. These include performance, maintainability, safety, reliability, and security.
 - **Example:** “The password reset must happen within 2 minutes.”

8. Emergent Properties

- Properties that arise only from the interaction of all components.
- They cannot be satisfied by a single module.
- Security is a major emergent property.
- **Example:** Call center throughput depends on the telephone system, the software, and the operators working together.

9. Software Design

- The bridge between requirements and construction.
- **Activities:**
 - **Architectural Design (High-level):** Defines components and their structure.
 - **Detailed Design:** Defines each component's logic.
- **Principles:**
 - Abstraction
 - Modularity & Decomposition
 - Encapsulation (hiding internals)
 - Cohesion (keeping related functions together)
 - Coupling (minimizing dependencies)
 - Separation of concerns
- **Design for Security:**
 - Prevent unauthorized access.
 - Tolerate attacks by limiting damage and enabling recovery.
- **Example:** A microservices architecture where each service has a limited scope, resulting in low coupling and high cohesion.

10. Software Construction

- The process of building working software, which includes coding, testing, and debugging.
- **Closely tied with:**
 - Testing (unit + integration tests)
 - Quality (ensuring correctness)
- **Example:** Writing code for a payment gateway, including test cases to check for successful and failed transactions.

11. Software Testing

- The dynamic verification that a program behaves as expected.
- **Key Features:**

- **Dynamic:** The system state matters.
- **Finite:** It is impossible to test everything, so test cases must be selected.
- **Selected:** Test cases should be selected based on risk.
- **Expected:** Clear expected outcomes must be defined.
- **Example:** Testing a login module would include testing with valid input, an invalid password, and an SQL injection attempt.

12. Supporting Activities

a) Software Quality

- **Definitions:**
 - **Crosby:** Conformance to requirements.
 - **Humphrey:** Fitness for use.
 - **IBM:** Market-driven quality.
- **Cost of Quality Categories:**
 - **Prevention:** Training, process improvement.
 - **Appraisal:** Testing, reviews.
 - **Internal failures:** Fixes made before the product is released.
 - **External failures:** Fixes made after the product is released, which are the most costly.
- **Example:** Fixing a bug during the coding phase is cheaper than fixing it after the product has been released to the public.

b) Configuration Management (CM)

- The discipline of identifying and controlling versions/configurations at specific points in time.
- Ensures integrity and traceability.
- **Example:** Git version control tracks code changes and allows for easy rollbacks.

c) Software Maintenance

- Software continues to evolve after release due to defects, environment changes, and new requirements.
- **Maintainer's Activities:**
 - Day-to-day control.

- Perfecting existing functions.
 - Applying security fixes.
 - Preventing performance degradation.
- **Example:** WhatsApp adding new privacy features via software updates.

d) Software Engineering Management

- **Management:** Planning, monitoring, controlling, and reporting.
- **Challenges:**
 - Clients may not know their exact needs.
 - Requirements can change.
 - High novelty and complexity.
 - Rapid technological evolution.
- **Example:** Managing AI projects where requirements evolve as machine learning models improve.

e) Software Security

- Security must be embedded across the entire software lifecycle.
- Vulnerabilities can come from not only the code, but also from:
 - Libraries
 - **COTS** (Commercial Off-the-Shelf software)
 - OS
- **Example:** The Log4j vulnerability (2021) came from a popular library.

f) Software Engineering Economics

- Relates software to economic measures.
- **Focus:** Risks, ROI, costs, and benefits.
- **Example:** Choosing between an open-source vs. a proprietary database for cost savings.

g) Professional Practice

- The ethical, responsible, and professional side of software engineering.
- Governed by the **IEEE-CS/ACM Code of Ethics** (1999), which has 8 principles:
 - Public, Client, Product, Judgment, Management, Profession, Colleagues, Self.

- **Example:** A developer refusing to insert a backdoor into an app, demonstrating their ethical responsibility.

Summary of Session 2

- **Software Engineering:** A disciplined approach to building software.
- **SWEBOK v4:** Expanded to include new knowledge areas like Security and Architecture.
- **Requirements:** Include both Functional and Non-functional types.
- **Design Principles:** Emphasize modularity, abstraction, cohesion, and coupling.
- **Supporting Activities:** Quality, CM, and Maintenance are just as crucial as core development.
- **Economics & Ethics:** Play a central role in professional practice.
- **Security:** Must be a built-in property across the entire software lifecycle.

Secure Software Engineering - Session 3

1. Work Products During the SDLC

Software engineering produces models and artifacts at each stage of the Software Development Life Cycle (SDLC).

Types of Models:

- **Requirement Models:** Define what the system should do.
 - **Examples:** Requirement documents, user stories, use cases.
- **Scenario-based Modeling:** Shows the system from a user's perspective.
 - **Example:** A student logs into a portal to register for a course.
- **Flow-oriented Modeling:** Explains how data moves through the system and is processed.
 - **Example:** Online payment → Input card details → Process payment → Generate receipt.
- **Class-based Modeling:** Defines objects, attributes, and relationships.
 - **Example:**
 - **Class:** User
 - **Attributes:** Name, Email, Password
 - **Relationship:** User makes → Payment.
- **Behavioral Modeling:** Shows states and events that change them.
 - **Example:** ATM states: Idle → Card Inserted → PIN Entered → Transaction Processing → Cash Dispensed.

2. Case Study: SafeHome System (Pressman)

The **SafeHome** system is a home security system.

- **Functions:** Detect fire, flood, gas leaks, intrusions, etc.
- **Uses:** Wireless sensors and a control panel.
- **Action:** Automatically alerts a monitoring agency.

Objects in SafeHome

Control panel, smoke detectors, motion sensors, alarm, event log, display, PC, telephone call.

Services in SafeHome

Configure system, set alarm, monitor sensors, dial phone, display status.

3. Use Cases

- **Definition:** A collection of user scenarios describing interactions with the system.
- **Actor:** A person or device interacting with the system.
- **Example: SafeHome "Initiate Monitoring"**
 - **Actors:** Homeowner, sensors, technicians.
 - **Flow:**
 1. Homeowner enters password.
 2. Selects "stay" or "away".
 3. System arms sensors.
 - **Precondition:** Control panel is ready.
 - **Postcondition:** Sensors are activated.

4. Misuse Cases

- **Definition:** Describes how an attacker may exploit the system.
- **Purpose:** Complements use cases to identify security weaknesses.
- **Example: Attack on "Forgot Password"**
 - **Possible Attacks:**
 - SQL injection
 - Brute force guessing emails
 - Sniffing password emails
 - **Worst Case:** Attacker steals data or resets the password to their own email.
 - **Mitigation:**
 - Use parameterized queries.
 - Lock accounts after failed attempts.
 - Encrypt transmission (TLS/SSL).

5. Data Flow Modeling

- **Data Flow Diagram (DFD):** Shows how input is transformed into output.
- **Guidelines:**
 - Start with Level 0 (context diagram) → one big bubble.
 - Identify inputs, outputs, and data stores.
 - Refine into more detailed levels (Level 1, Level 2).
- **Example:** Online Shopping → Input: Order request → Process: Validate + Payment → Output: Confirmation.

6. Security Implications in SDLC – OWASP SAMM

- **SAMM (Software Assurance Maturity Model):** An OWASP project that helps organizations measure and improve their software security posture.
- Covers the entire SDLC and is technology/process agnostic.
- Structured into 15 practices under 5 business functions.
- **Example:**
 - **Governance:** Policies and compliance.
 - **Construction:** Secure coding practices.
 - **Verification:** Testing, audits.

7. Security Development Lifecycle (SDL – Microsoft)

- **SDL:** Microsoft's process to reduce vulnerabilities.
- **Core Concepts:**
 - **Education:** Train developers on secure coding.
 - **Continuous Improvement:** Learn from past vulnerabilities.
 - **Accountability:** Assign roles for security.
- **Practices:**
 - **Threat Modeling:** Identify threats early.
 - **Attack Surface Reduction:** Limit exposed entry points.
 - **Static & Dynamic Analysis:** Use tools to check code quality.
 - **Fuzz Testing:** Inject random or malformed inputs.
 - **Final Security Review:** Approval before release.

8. BSIMM – Building Security In Maturity Model

- Unlike SDL, **BSIMM** is an observational model based on the study of industry practices.
- Based on 12 practices across 4 domains: Governance, Intelligence, SSDL Touchpoints, and Deployment.
- **Examples of Companies Studied:** Adobe, Google, Microsoft.
- **Key Terms:**
 - **SSG (Software Security Group):** A dedicated security team.
 - **Champions:** Developers trained as local security advocates.
 - **SSI:** Enterprise-wide security initiative.

9. Security in Agile Development

- Agile values working software and fast delivery.
- **Challenge:** Security is often ignored due to the focus on speed.
- **Best Practices:**
 - Think like a hacker and train developers on vulnerabilities.
 - Consider malicious users and write security user stories.
 - Continuous Integration with security scans.
 - Peer code reviews in every iteration.
 - Innovate security tools customized for agile workflows.
 - Cultivate a security culture through training and awareness.
- **Scenario:** In a sprint, add misuse stories (e.g., “Attacker tries SQL injection”) alongside user stories.

10. DevOps and DevSecOps

- **DevOps:** Combines development and operations for faster releases.
 - **Practices:** CI/CD, Infrastructure as Code, Monitoring.
- **DevSecOps:** Integrates security into the DevOps pipeline.
 - **Shift Left:** Apply security checks earlier in the SDLC.
 - **Shared responsibility:** Development, Security, and Operations teams work together.
- **Example:**
 - Automated static code analysis runs in every CI build.

- Vulnerability scans are performed before deployment.

11. Challenges in DevSecOps

- Lack of security skills among developers.
- Organizational barriers (e.g., Dev vs. Ops mindset).
- Security not being treated as a priority.

12. Low-Code/No-Code Security Risks

- **OWASP** lists risks in low-code platforms (e.g., OutSystems).
- **Examples:**
 - Weak authentication defaults.
 - Improper data storage.
 - Misconfigurations.

Summary

- This session connected requirements engineering models (use cases, misuse cases, DFDs) with their security implications.
- We introduced security maturity models (**SAMM**, **SDL**, **BSIMM**).
- We also showed how to integrate security into modern methodologies (**Agile**, **DevOps**, **DevSecOps**).
- The key takeaway is the **shift-left approach**, which involves bringing security into the development process as early as possible.

Secure Software Engineering - Session 4

Securing a Computer-Based System

A computer system's security is defined by its weakest point. This is known as the **Principle of Easiest Penetration**, meaning attackers will always find the easiest way to get in, not necessarily the most obvious one.

Assets

- Hardware
- Software
- Data

Vulnerabilities

Weaknesses that can be exploited.

- **Example:** A file server without authentication.

Threats

A potential loss or harm that might occur.

- **Example:** Publicly exposed user files.

Threat Modeling

A **threat model** defines what assets are being protected, who they are being protected from, and what needs to be prevented.

Threats to Assets (Pfleeeger)

- **Interruption:** The destruction or unavailability of an asset, which affects **availability**.
- **Interception:** Unauthorized access to an asset, which affects **confidentiality**.
- **Modification:** Unauthorized tampering with an asset, which affects **integrity**.
- **Fabrication:** The insertion of counterfeit objects, which affects **authenticity**.

Threat Consequences (RFC 4949)

- **Unauthorized Disclosure:** Exposure, interception, inference, intrusion.
- **Deception:** Masquerade, falsification, repudiation.
- **Disruption:** Incapacitation, corruption, obstruction.
- **Usurpation:** Misappropriation, misuse.

Attackers

- **Amateurs:** Employees, disgruntled users, accidental discoverers.
- **Crackers/Malicious Hackers:** Seek unauthorized access for fun, curiosity, or gain.
- **Career Criminals:** Professionals seeking financial rewards.
- **Terrorists:** Use computers as targets, propaganda, or attack platforms.

The M-O-M Triad

An attack requires a combination of three elements. Removing any one of them can prevent the attack.

Motive+Opportunity+Method→Attack

- **Motive:** The reason for the attack.
- **Opportunity:** The time and access needed.
- **Method:** The skills and tools required.

Defense Methods

- **Prevent:** Stop the attack from happening.
- **Deter:** Discourage the attacker.
- **Deflect:** Misdirect the attack.
- **Detect:** Find out when an attack is happening.
- **Recover:** Restore the system after an attack.
- **Example (Car Theft):** An immobilizer prevents theft, secure parking deters, an alarm detects, and insurance helps you recover.

Structured Approaches to Threat Modeling

Adam Shostack's Approach

1. What are you building?
2. What can go wrong?
3. What should you do about it?
4. Did you analyze it properly?

SDL Threat Modeling (Microsoft)

- **Objectives:** Secure by design, identify threats early, and improve predictability of security.
- **Responsibilities:** The Project Manager owns the process, developers create diagrams, and testers identify threats.
- **Process:**
 1. Define security requirements.
 2. Create application diagrams (DFDs).
 3. Identify threats.
 4. Mitigate threats.
 5. Validate that threats are mitigated.

STRIDE Threats

A model for classifying threats based on the security property they violate.

- **Spoofing** Breaks authentication
- **Tampering** → Breaks integrity
- **Repudiation** → Breaks accountability
- **Information Disclosure** → Breaks confidentiality
- **Denial of Service** → Breaks availability
- **Elevation of Privilege** → Breaks authorization

Standard Mitigations

- **Spoofing:** Authentication, Digital signatures.
- **Tampering:** Integrity checks, Access Control Lists (ACLs).
- **Repudiation:** Logging, Auditing.
- **Information Disclosure:** Encryption, ACLs.
- **DoS:** Filtering, Quotas.
- **EoP:** Role-based access, Input validation.

DREAD Threat-Risk Model

DREAD Factors:

- **Damage** – How big is the damage if attack succeeds?
- **Reproducibility** – How easy is it to reproduce the attack?
- **Exploitability** – How much effort/expertise is needed?
- **Affected Users** – What % of users will be affected?
- **Discoverability** – How easy is it to find the threat?

Scoring:

Each factor is rated (e.g., 1–10). Final risk score = Average of all 5.

Example – College Library Website

Threat: Malicious user views confidential data.

- **Damage:** 8 (high financial/reputation/legal impact)
- **Reproducibility:** 10 (fully reproducible)
- **Exploitability:** 7 (needs same subnet/router access)
- **Affected Users:** 10 (all users impacted)
- **Discoverability:** 10 (easy to find)

Overall DREAD Score = $(8 + 10 + 7 + 10 + 10) / 5 = 9$ (High Risk)

Threat Modeling Frameworks

1. MITRE ATT&CK

A publicly accessible knowledge base of **adversarial tactics, techniques, and common knowledge**. It's used to develop threat models and defenses by showing how attackers operate in a matrix format.

- **Tactics:** The "why" of an attack (e.g., Initial Access, Execution, Defense Evasion, Credential Access, Lateral Movement).
- **Example (Web App Attack):** An attacker gains **Initial Access** to a client via the web app, **Executes** malicious code, steals **Credentials**, and performs **Lateral Movement** to a database to steal data.

2. Lockheed Martin Cyber Kill Chain

This model defines the distinct stages an attacker goes through to achieve their objective.

- **Reconnaissance:** Researching the target.
- **Weaponization:** Creating the malicious payload.
- **Delivery:** Transmitting the payload (e.g., phishing email).
- **Exploitation:** The payload executes and exploits a vulnerability.
- **Installation:** The malware or backdoor is installed.
- **Command & Control (C2):** The attacker gains remote access to the system.
- **Actions on Objective:** The attacker achieves their goal (e.g., data theft).

3. Privacy Threat Modeling – LINDDUN

LINDDUN is a framework for systematic privacy threat modeling, developed at KU Leuven. It helps identify and mitigate privacy risks.

- **Linkability:** The ability to link two actions of a user.
- **Identifiability:** The ability to identify a person from data.
- **Non-Repudiation:** The user cannot deny an action (opposite of desired privacy).
- **Detectability:** Being able to detect if a record exists.
- **Disclosure of Information:** Unauthorized data exposure.

- **Unawareness:** Users are not informed of privacy risks.
- **Non-Compliance:** Not following data protection laws like GDPR.

4. More Methods of Threat Modeling

Other approaches to threat modeling include:

- **Attack Trees:** A graphical model of an attacker's goals and paths.
- **OCTAVE:** A framework for evaluating operational threats and vulnerabilities.
- **PASTA:** A process for attack simulation and threat analysis.
- **VAST:** A Visual, Agile, and Simple Threat modeling framework.

Security Requirements Engineering - Session 5

What is a Requirement?

A requirement is a need, expectation, constraint, or interface of a stakeholder.

- **Need:** A basic necessity.
- **Expectation:** An unstated but assumed requirement.
- **Constraint:** An unavoidable limitation.
- **Interface:** Interaction with users, suppliers, or peers.

Stakeholders include end users, the project team, management, and marketing.

Importance of Requirements Engineering

Fixing requirement errors later is 10–200× costlier. Poor requirements can lead to projects that are over budget, delivered late, have poor quality, or result in an unused product.

Requirements Engineering Process

1. **Inception:** Understand the problem and stakeholders.
2. **Elicitation:** Gather requirements from stakeholders.
3. **Elaboration:** Build an analysis model (data, function, behavior).
4. **Negotiation:** Agree on realistic deliverables.
5. **Specification:** Document requirements using text, models, use cases, or prototypes.
6. **Validation:** Check for errors, gaps, and inconsistencies.
7. **Management:** Track, prioritize, and handle changes.

Eliciting Requirements

Conduct structured meetings with a facilitator to gather requirements.

- **Techniques:** Use cases, FAST meetings, QFD (Quality Function Deployment), and prioritization.

Quality Function Deployment (QFD)

- **Definition:** A method developed by Dr. Yoji Akao in to capture the voice of the customer and translate it into technical requirements.
- **Goal:** To ensure that customer needs are reflected in system and software functions.
- **Key Points:**
 - **Function Deployment:** Finds the value of each function from the customer's perspective.
 - **Types of Requirements:**
 - **Normal:** Directly stated by the customer.
 - **Expected:** Assumed, but not explicitly stated.
 - **Exciting:** Extra features that delight customers.
 - **Information Deployment:** Identifies data objects and events.
 - **Task Deployment:** Studies system behavior.
 - **Value Analysis:** Prioritizes requirements based on their importance.

Requirements Challenges

Requirements are often ambiguous, incomplete, inconsistent, and infeasible. Non-functional requirements (quality and security) are frequently missed.

Security Requirements

Security is not a one-time task; it must adapt to the evolving environment and goals. It is an emergent property that must be built from the ground up, not patched on later.

Security Thinking

To think like an attacker, use **misuse cases** (undesired behaviors) and **abuse cases** (harmful results of misuse).

- **Ask:**
 - How can good input be distinguished from bad input?
 - Where might attackers be (network, workstation, backend)?
 - What can attackers do (e.g., eavesdrop, modify traffic, change configuration)?

CMU SQUARE Process

The **SQUARE** (Security Quality Requirements Engineering) process provides a structured approach to defining security requirements.

- **Steps:**
 1. Agree on definitions.
 2. Identify security goals.
 3. Develop artifacts (scenarios, misuse cases).
 4. Perform a risk assessment.
 5. Select elicitation techniques.
 6. Elicit security requirements.
 7. Categorize requirements (system, software, constraints).
 8. Prioritize requirements.
 9. Inspect.

Sample: Identify Security Goals (AMS Example)

- **System:** Asset Management System (AMS)
- **Business Goal:** Provide a reliable application for managing assets.
- **Security Goals:**
 - Ensure management control over configuration and usage.
 - Protect the confidentiality, accuracy, and integrity of data.
 - Keep the system available whenever needed.

1. Attack Patterns

- **Definition:** Attack patterns describe common ways attackers exploit software. They capture the attacker's perspective to help in identifying and mitigating security risks.

- **Key Components:** Pattern Name, Prerequisites, Description, Related Vulnerabilities, Method of Attack, Attacker Goal, Skill Level, Resources, Mitigations, Context, and References.
- **Example: Shell Command Injection**
 - **Prerequisites:** The application passes user input directly into shell commands.
 - **Description:** A user inputs `username.dat; rm -rf /;` into a web form. The shell executes both `cat data_log_username.dat` and the destructive `rm -rf /`.
 - **Diagram Idea:**

User Input --> Application --> Shell Command Execution --> System Response

|

+--> Malicious input triggers unwanted commands

•

2. Threat Tree

- **Definition:** Threat trees are hierarchical diagrams that represent potential security threats, starting from a high-level attack goal and breaking it down into detailed attack paths.
- **Example:** Unauthorized access to an Asset Management System (AMS).

Unauthorized Access

| — Weak Passwords

| | — Dictionary Attack

| | — Brute Force

| — Poor Account Management

| | — Default Admin Account

| | — Privilege Escalation

└─ OS Vulnerabilities

└─ Unpatched OS

└─ Exploit Remote Services

-

3. Use Case vs. Abuse Case

- **Use Case:** Describes the intended behavior of a system (e.g., A user logs in with a password).
- **Abuse Case:** Describes unwanted scenarios from the perspective of an attacker (e.g., A disgruntled employee attempts to break the password).
- **Example:** Dictionary Attack
 - **Abuse Case:** A disgruntled employee attempts to use a dictionary attack to break a password.
 - **Diagram Idea:**

Actor: Disgruntled Employee

|

v

Abuse Case: Break Password

└─ Step 1: Exploit weak account management

└─ Step 2: Attempt dictionary attack

└─ Step 3: Access confidential data

4. Risk Assessment

- **Definition:** The process of assessing potential threats and their impact on system assets to prioritize mitigation strategies.

- **Example (AMS):**
 - **Assets:** Windows Server, Sybase databases.
 - **Threats:** Unauthorized access, data loss, tampering.
 - **Impact:** Loss of decision-making ability.
- **Diagram Idea: Risk Matrix**

Likelihood

|H | High Risk (Critical)

|M | Medium Risk

|L | Low Risk

+----- Impact

L M H

5. Elicitation Methods for Security Requirements

- **Methods:**
 - **Misuse/Abuse Cases:** Identify negative scenarios an attacker might attempt.
 - **Quality Function Deployment (QFD):** Translate customer needs into technical requirements.
 - **Joint Application Development (JAD):** Structured meetings with stakeholders to gather and validate requirements.

6. SQUARE Process in SDLC

- **SQUARE:** Security Quality Requirements Engineering
- **Integration:** Security requirements are incorporated into the **SRS** (Software Requirements Specification) alongside functional requirements.

7. OWASP SAMM (Security Assessment Maturity Model)

SAMM is divided into five business functions, with security requirements being a core part of the **Design** function.

- **SR1 – Consider Security Explicitly:** Security should be a clear part of the requirements process.
 - **Example:** For an AMS, requirements should explicitly state the need for strong authentication and data integrity.
- **SR2 – Increase Granularity of Security Requirements:** Security requirements should be detailed and based on business logic and known risks.
 - **Example:** In an AMS, SR2 ensures that only users with the "Finance Manager" role can approve asset transfers.
- **SR3 – Mandate Security Requirements Process:** Security requirements must be mandatory for all software projects and third-party dependencies.
 - **Example:** Every new AMS module must pass a security checklist before release.
- **Assessment Matrix for Security Requirements:** SAMM uses a scoring matrix to evaluate the maturity of a company's security requirements process, from **no** adoption to **most/all** adoption.

8. Security Development Lifecycle (SDL)

- **Requirements Phase:** Security and privacy are considered at the project's inception.
- **Key Practices:**
 - Establish security and privacy requirements.
 - Create quality gates and bug bars to define acceptable vulnerability thresholds.
 - Perform security and privacy risk assessments.
- **SDL for Agile:**
 - **Every-sprint requirements:** Performed in every sprint (e.g., threat modeling, daily security scans).
 - **Bucket requirements:** Regular but not mandatory in every sprint (e.g., a formal design review).
 - **One-time requirements:** Performed once per project (e.g., setting up a security advisor).

Secure Software Engineering - Session 6

1. Software Design (SWEBOK Definition)

Software design is the activity that translates requirements into a structure that can be built. It has two parts:

- **Architectural Design (High-level):** Defines the overall system structure, its main components, and their relationships.
- **Detailed Design (Low-level):** Specifies each component in detail for implementation.

Example: Designing a house: Architectural design is the blueprint, while detailed design covers the wiring and plumbing.

2. Understanding Software Architecture

Architecture is the structure of a system, consisting of components (modules, services), their externally visible properties (interfaces), and their relationships.

Example: In an e-commerce system, the components are the payment service, cart service, and order service. Their relationships are how they connect and interact, such as the cart linking to checkout, which links to payment.

3. Importance of Architecture

Architecture serves as a communication tool for all stakeholders. Early design decisions significantly affect a project's success. For example, deciding on a microservices-based vs. monolithic architecture early on impacts everything from deployment to scalability.

4. Uses of Architecture

- **Reuse:** Promotes modular development.
- **Construction & Evolution:** Allows work to be divided among teams.
- **Analysis:** Helps predict performance and security before development begins.

5. Objectives of Software Architecture & Design

- **Completeness:** Covers all requirements.
- **Stability:** Works consistently in its context.
- **Flexibility & Extensibility:** Can adapt to new changes and future growth.
- **Scalability:** Handles increasing load.

6. Security-Specific Objectives

- **Comprehensive Security Architecture:** All security features are active.
- **Attack Resistance:** Minimizes exploitable weaknesses.
- **Attack Tolerance:** The software continues to function during an attack.
- **Attack Resilience:** Limits damage if an attack succeeds.

Example: During a DDoS attack on an e-commerce site, non-essential features may be disabled, but the checkout process remains functional.

7. Secure Architecture Risk Analysis

A **secure architecture** is resilient and adaptable, not just a collection of security features. A risk assessment minimizes risks by considering threats and technology.

- **Steps:**
 1. **Software Characterization:** Define boundaries and requirements.
 2. **Threat Analysis:** Identify possible threats.
 3. **Vulnerability Assessment:** Check for weaknesses.
 4. **Likelihood & Impact:** Determine the probability and potential damage.
 5. **Mitigation:** Apply countermeasures.

8. Architectural Vulnerability Analysis

- **Known Vulnerability Analysis:** Compare your design against best practices (e.g., avoid running services as root).
- **Ambiguity Analysis:** Identify unclear requirements (e.g., how to handle an account lock).
- **Platform Vulnerability Analysis:** Check for weaknesses in the OS and network environment.

9. Principles for Secure Design

- **Core Concepts:**
 - **Abstraction:** Hide details (APIs).
 - **Separation of Concerns:** Divide problems into manageable parts.
 - **Modularity & Information Hiding:** Use independent parts with controlled interfaces.
 - **Least Privilege:** Give minimal access rights.

10. Secure Design Principles

- **Securing the Weakest Link:** Attackers will target the easiest point.
- **Defense in Depth:** Use multiple security layers.
- **Failing Securely:** The system must remain secure even during a failure (e.g., deny access if the login service crashes).
- **Least Privilege:** Run with minimal rights (e.g., a web server running as a limited user).
- **Separation of Privilege:** Requires multiple conditions for an action (e.g., two keys to launch a missile).
- **Economy of Mechanism:** Keep the design simple to make it easier to secure.
- **Least Common Mechanism:** Avoid shared access in multi-user apps.
- **Reluctance to Trust:** Always assume the environment is insecure and validate user input.
- **Complete Mediation:** Check access every single time, not just once.
- **TOCTOU (Time of Check to Time of Use):** A race condition where an attacker changes a file after a permission check but before its actual use.

- **Psychological Acceptability:** Security should be usable so users don't find workarounds.
- **Promoting Privacy:** Do not expose user data unnecessarily.

11. Secure Architectural Patterns

These are high-level patterns for securing the overall system architecture.

- **Distrustful Decomposition:** Break the system into untrusted chunks (e.g., browser plugins running in a sandbox).
- **Privilege Separation (PrivSep):** Separate privileged code from unprivileged code (e.g., the SSH authentication vs. session).
- **Defer to Kernel:** Use OS-level security features instead of writing custom ones.

Classes of Secure Design Patterns

Security design patterns are structured solutions that address recurring security issues. They are classified into **Design-level** and **Implementation-level** patterns.

1.1 Design-level Patterns

These address internal design issues of high-level components.

- **Secure Factory:** Controls object creation to avoid unsafe objects.
 - **Example:** A factory that creates database connections ensures every connection uses strong authentication and encryption (TLS).
- **Secure Strategy Factory:** Dynamically and securely selects algorithms.
 - **Example:** A payment system securely selects encryption algorithms (AES-, RSA) based on the context, preventing the use of weak algorithms.
- **Secure Builder Factory:** Builds complex objects step-by-step, ensuring each step follows security rules.
 - **Example:** While building a user profile object, the builder ensures passwords are hashed, not stored in plain text.
- **Secure Chain of Responsibility:** Passes requests through a chain of handlers with security checks.

- **Example:** A financial transaction request passes through fraud detection, authentication, and authorization before execution.
- **Secure State Machine:** Manages system states safely and prevents illegal transitions.
 - **Example:** A login system cannot transition directly from "locked out" to "logged in" without proper authentication.
- **Secure Visitor:** Adds secure operations to objects without altering their classes.
 - **Example:** Security audits visit different components (DB, files, APIs) to check compliance without changing the components themselves.

1.2 Implementation-level Patterns

These address low-level coding issues and align with CERT Secure Coding Standards.

- **a) Secure Logger**
 - **Problem:** Logs can reveal sensitive data (IPs, stack traces).
 - **Solution:** A **Secure Logger** sanitizes logs to prevent data leakage and ensures logs are tamper-proof.
 - **Example:** Instead of logging "Login failed: password=secret", secure logging shows only "Login failed for user John".
- **b) Clear Sensitive Information**
 - **Problem:** Sensitive data may remain in memory or cache after use.
 - **Solution:** Always wipe or clear sensitive data from memory before releasing resources.
 - **Example:** After processing a payment, the memory storing the credit card number should be overwritten with zeros.
- **c) Secure Directory**
 - **Problem:** An attacker can tamper with files in insecure directories.
 - **Solution:** A program should only read and write in **secure directories** with proper permissions.
 - **Example:** File uploads are stored in `/secure/uploads/` with restricted access, not in `/tmp` which is shared.
- **d) Input Validation**
 - **Problem:** Unvalidated inputs can lead to major attacks (SQL injection, XSS).
 - **Solution:** Validate all external inputs, use whitelists, and perform server-side validation.

- **Example:** Reject " OR 1=1 --" and only allow alphanumeric usernames in a login form.

2. Software Composition Analysis (SCA)

Modern applications rely heavily on open-source components, which can introduce risks.

- **What SCA Does:** SCA tools scan the codebase to identify all open-source dependencies (direct and transitive) and check for license compliance and security vulnerabilities (CVEs).
- **Functions of SCA Tools:**
 - **Detection:** Identify vulnerable libraries (e.g., detecting Log4j v2.14.1).
 - **Prioritization:** Assess the criticality of vulnerabilities.
 - **Remediation:** Suggest patches or upgrades.
- **Direct vs. Transitive Dependencies:** A **direct dependency** is a library you explicitly import (e.g., `spring-core`), while a **transitive dependency** is a library that is pulled in indirectly by another library (e.g., `commons-logging` pulled in by `spring-core`).

3. Log4j & Risks of Open Source Software

The **Log4j vulnerability** (2021, CVE-2021-44228) allowed Remote Code Execution (RCE) via crafted log messages, affecting millions of applications. SCA tools are crucial for detecting such vulnerabilities in both direct and transitive dependencies.

Summary Table

Category	Pattern/Concept	Problem Solved	Example
Design-level	Secure Factory	Safe object creation	DB connection factory ensures encryption

	Secure Chain of Responsibility	Sequential security checks	Transaction passes fraud/auth checks
Implementation-level	Secure Logger	Prevent log misuse	Hide passwords in logs
	Clear Sensitive Info	Remove data from memory/disk	Wipe credit card memory
	Secure Directory	Prevent file tampering	Restrict uploads to secure folder
	Input Validation	Block injection attacks	Validate username input
SCA	Dependency Scan	Detect vulnerable libs	Identify Log4j v2.14.1
	Remediation	Fix issues via patch	Upgrade Log4j to v2.17.1
Risk (OSS)	Vulnerable library use	Exploitable apps	Log4Shell incident

Secure Software Engineering – Session 7

1. Testing Strategy

- **Definition:** A *testing strategy* is a roadmap for testing activities. It explains *what to test*, *when to test*, *how much effort/resources to spend*, and *how tests should be executed*.
- **Process:**
 - **Plan tests** (define objectives, tools, resources, schedule).
 - **Design test cases** (normal & abnormal cases).
 - **Execute tests** (run and record results).
 - **Evaluate results** (check against expectations).
- **Scope:**
 - Begins at **component level** (unit testing).
 - Progresses to **integration testing** and **system testing**.
 - Uses **different techniques at different stages**.

♦ *Example:*

For an online banking app:

- Component level → test “login” function with valid & invalid credentials.
- Integration → test if “login” + “fund transfer” work together.
- System → test the entire workflow: login → transfer → confirm → logout.

2. Software Testing Axioms (Ron Patton, 1990s)

1. **Impossible to test completely** → Too many possible inputs/paths.
2. **Testing is risk-based** → Prioritize critical areas (e.g., payment systems).
3. **Testing cannot prove absence of bugs** → Only shows presence of bugs.
 - *Pesticide Paradox*: The more you use the same tests, the less effective they become (software becomes “immune”).
 - Need to create new test cases regularly.

4. **More bugs found** → **more still exist** → Bugs cluster.
5. **Not all bugs will be fixed** → Depends on priority & cost.
6. **Defining a “bug” can be tricky** → Sometimes requirement is unclear.
7. **Specifications are never final** → They evolve over time.
8. **Testers may be unpopular** → They expose problems.
9. **Testing is a technical discipline** → Requires skill, tools, and methodology.

♦ *Example:*

If you test an e-commerce site repeatedly with the same dataset, it may pass. But attackers could find new injection points. That's why rotating test data & attack patterns is needed.

3. Key Issues in Testing (SWEBOK)

- **Dynamic:** Output depends not just on input but also on *system state*.
Example: Retry login after 3 failures → account lock.
- **Finite:** Too many test cases possible, exhaustive testing impractical.
- **Selected:** Must choose best test set (risk-based).
- **Expected:** Need a clear pass/fail criterion.

4. Security Testing

- **Goal:** Ensure **confidentiality, integrity, availability (CIA)** of system and data.
- **Includes:**
 - Verification against **misuse/abuse**.
 - **Negative testing:** test abnormal, malicious inputs.

♦ *Example:*

- Confidentiality → Ensure passwords are encrypted.
- Integrity → Ensure transferred funds are not altered.
- Availability → Ensure system resists DDoS attack.

5. Myth vs Reality in Security Testing

- **Myth:** Security testing is a “black art” – only for experts with deep knowledge.
- **Reality:** Even testers with limited security expertise can contribute using tools & structured methods.

6. Penetrate & Patch Approach

- Tools (reactive, post-release):
 - **Nmap** → Scans open ports.
 - **Burp Suite Intruder** → Automates web app attacks.
 - **Metasploit** → Exploit known vulnerabilities.
- **Problem:** Only fixes after discovery. A patch may close one hole but open another.
- **Better:** Integrate security testing into development lifecycle.

7. Window of Vulnerability (OWASP)

- **Timeline:**
 1. Vulnerability discovered.
 2. Vendor notified.
 3. Vulnerability published.
 4. Tools updated (e.g., IDS signatures).
 5. Patch released.
 6. Patch installed on all systems.
- **Risk:** Systems remain exposed between discovery and patch installation.

♦ *Example:*

- Heartbleed bug (OpenSSL) → discovered, published → patch released → many sites delayed patching → attackers exploited.

1. Source Code Analyzers

What are they?

- Automated tools that scan source code to detect **security vulnerabilities**.
- They use **data-flow** and **control-flow** analysis to detect subtle bugs.
- Can be tuned:
 - **More sensitive** → catches more bugs but raises more false alarms.
 - **Less sensitive** → fewer false alarms but might miss real issues.

Capabilities

1. Check insecure library calls

- Many vulnerabilities come from unsafe functions like `strcpy()` in C.
- Tools maintain a **database of unsafe functions**.
- They also preprocess code (so macros/headers are expanded) and tokenize it.

✓ Example:

```
char buffer[10];
strcpy(buffer, "This string is way too long!"); // buffer overflow
```

2. Analyzer flags: *Potential buffer overflow using strcpy()*.

2. Bounds Checking Errors / Scalar Type Confusion

- **Integer overflow:** `int x = 2147483647 + 1; // becomes negative`
 - **Integer truncation:** Assigning a long value to a short.
 - **Unsigned underflow:** `unsigned int u = 0; u--; // becomes very large`
3. 👉 Vulnerable if used for **buffer sizes**.
-

3. Type Confusion in References/Pointers

- When a pointer/reference is cast to the wrong type.

Example in C++:

```
class A { virtual void foo(); };
class B { virtual void bar(); };

A* a = new A();
B* b = (B*)a; // Wrong cast → runtime confusion
b->bar(); // Undefined behavior, possible vulnerability
```

○

4. Memory Allocation Errors

- Common in C/C++ heap usage.
- Issues:
 - **Buffer overflow in allocated memory**
 - **Double free**
 - **Use-after-free**

👉 Example:

```
char* data = malloc(10);
free(data);
free(data); // double free
```

5.

5. Control Flow Vulnerabilities

- Wrong order of operations → security risk.

Example: **TOCTOU (Time-of-check to time-of-use)**

```
if (access("file.txt", W_OK) == 0) {
    // File is writable at this moment
```

```
fd = open("file.txt", O_WRONLY);
write(fd, "data", 4);
}
```

- - Attacker can replace `file.txt` between the check & write.

6. Data Flow Analysis

- Tracks how **tainted data** (user input) flows through the program.
- If unchecked → vulnerability.

✓ Example:

```
char input[100];
gets(input); // user-controlled
printf(input); // format string vulnerability
```

7.

7. Pointer Aliasing

- When two pointers point to the same memory location.
- Modifying one indirectly modifies the other.
- Hard for analyzers to track.

Weaknesses of Source Code Analyzers

- Many false positives.
- Cannot detect **auth, access control, cryptography misuse** easily.
- Struggle with **configuration issues**.
- Hard to analyze incomplete/uncompilable code.

2. White Box Security Testing

- Tester has **access to source code**.
 - Usually done during **unit testing** phase.
 - Tester must:
 1. Understand secure vs insecure code.
 2. Think like an attacker.
 3. Use specialized tools.
-

Techniques

1. Data-Flow Analysis

- Trace variable definitions → usage.

Example:

```
int size;  
scanf("%d", &size); // user controls size  
char* buf = malloc(size); // risky allocation
```

- Analyzer tracks flow → *size depends on untrusted input*.
-

2. Fault Injection

- Modify program code to simulate errors.

Example: Insert extra code to force errors:

```
if (x == 0) crash(); // simulate divide-by-zero
```

-
-

3. Abuse Cases

- Thinking like an attacker:

- What if someone tries **SQL Injection**?
 - What if a user uploads a malicious file?
-

4. Trust Boundaries Mapping

- Identify **zones of trust** (e.g., web client ↔ server ↔ DB).
 - Ensure validation when data crosses boundaries.
-

5. Code Coverage Analysis

- Check if test cases execute all possible paths.
- Example: Path testing, statement coverage, condition coverage.

3. Black Box Security Testing

- Tester **does not see source code**.
- Focus on inputs, outputs, runtime behavior.

Goals

- Detect missed issues from white box testing.
- Find **config errors** (e.g., unhardened OS, wrong file permissions).
- Identify vulnerabilities due to **environment**.

Tools

1. Network Security Tools

- Example: **Nmap** (scans hosts, ports, services).
- Detects open ports (e.g., 22 SSH, 80 HTTP).

2. Database Security Tools

- Check DB misconfigurations (e.g., default passwords).
- Detect SQL injection risks.

3. Security Subsystem Tools

- Test cryptographic modules, random number generators.

4. Web Application Tools

- Focus on HTTP/HTTPS applications.
- Detect **XSS**, **SQLi**, **CSRF**.

4. Fuzz Testing

- Inject random or malformed data to see how system behaves.
- Originally: random noise generator (“fuzz” utility).
- Now: **smart fuzzing** → aware of protocols & formats.

Types of Fuzzing

1. **Black Box** – no knowledge of code, just random inputs.
2. **Grammar-Based** – generate inputs from protocol grammar.
3. **White Box** – generate inputs using source code (complex).

Variants

- **Network fuzzing**: Modify protocol messages.
- **File format fuzzing**: Corrupt files (e.g., JPEG, GIF overflow).
- **State-based fuzzing**: Send messages in wrong sequence.

👉 Example: Sending an HTTP **POST** before **CONNECT** request.

Handling Crashes

- Use **Address Sanitizer (ASAN)** to detect memory issues.
- Minimize crash cases for easier debugging.

Example: CERT BFF (Basic Fuzzing Framework)

- Used to find bugs in **Adobe Reader, Flash, Apple QuickTime, etc.**
- Uses machine learning to reduce manual effort.

Pros & Cons

✓ Advantages:

- Cheap, easy, finds severe bugs (e.g., crashes, buffer overflows).

✗ Issues:

- Takes a long time.
- Crashes are hard to analyze.
- Complex input formats need custom mutation logic.

5. Penetration Testing

Background

- Started in 1970s (Red Teams / Tiger Teams).
- Initially OS & network focused.
- Now extended to **applications**.

Key Points

- **Security is not compositional:** Two secure parts combined may still be insecure.
- Requires both **art (creativity)** and **science (tools & methodology)**.

Skills Needed

- Knowledge of domain (e.g., Web → HTTP, PHP, SQL).
- Knowledge of common attacks (SQL Injection, XSS).
- Familiarity with frameworks (e.g., Django, Rails).

Tools

1. **Host-Based** – check OS configs, file permissions.
2. **Network-Based** – scan for remote vulnerabilities. Example: **Nmap**.
3. **Application Proxies** – intercept/mod messages. Example: **Burp Suite**.
4. **Application Scanners** – automated SQLi/XSS testing.

Ethical Hacking Reminder

- Pen testing = **find vulnerabilities to fix**.
- Not to exploit for malicious intent.